Prefix Sums, 2D Prefix Sums & Difference Arrays

O(n) preprocessing, O(1) per query — and the reverse trick for updates

Learning goals

- Build prefix sums and answer range-sum queries in O(1)
- Generalize to XOR, counts, and any **invertible** operation
- Count subarrays by sum / remainder with prefix sums + hashing
- Build **2D prefix sums** with inclusion–exclusion
- Apply many range updates offline with **difference arrays** (1D and 2D)
- Know the limits: no updates between queries (→ Fenwick/segment trees later)

Motivation: CSES 1646 — Static Range Sum Queries

Array of n ≤ 2·10^5 values, q ≤ 2·10^5 queries "sum of a[l..r]".

- Naive: O(n) per query → 4·10^10 ×
- Prefix sums: O(n) once, then O(1) per query ✓

Definition

p[0] = 0, p[i] = a[1] + a[2] + ... + a[i] (1-indexed a)

```
i:     0  1  2  3  4  5
a:        3  1  4  1  5
p:     0  3  4  8  9  14
```

sum(l..r) = p[r] − p[l−1]

sum(2..4) = p[4] − p[1] = 9 − 3 = 6 = 1 + 4 + 1 ✓

The **extra p[0] = 0** removes every special case for l = 1.

Code

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    vector<long long> p(n + 1, 0);
    for (int i = 1; i <= n; i++) { long long x; cin >> x; p[i] = p[i-1] + x; }
    while (q--) {
        int l, r; cin >> l >> r;
        cout << p[r] - p[l - 1] << '\n';
    }
}
```

O(n + q). Use `long long`: 2·10^5 × 10^9 overflows `int`.

Beyond sums

Works for any operation with an **inverse**:

Operation	Prefix	Range query
sum	p[i] = p[i−1] + a[i]	p[r] − p[l−1]
xor	x[i] = x[i−1] ^ a[i]	x[r] ^ x[l−1]
count of property	c[i] = c[i−1] + [a[i] ok]	c[r] − c[l−1]
product mod prime	m[i] = m[i−1]·a[i]	m[r] · inv(m[l−1]) (Week 11)

Not min/max (no inverse) → sparse table (Week 24).

Example: CF 313B — Ilya and Queries

String s; queries (l, r): count i in [l, r) with s[i] = s[i+1].

```
// c[i] = number of positions j <= i with s[j] == s[j+1]   (1-indexed)
for (int i = 1; i < n; i++) c[i] = c[i-1] + (s[i-1] == s[i]);
// answer: c[r-1] - c[l-1]
```

Turn the property into a 0/1 array, then prefix-sum it.

Counting subarrays with sum x (negatives allowed)

sum(l..r) = x ⇔ p[r] − p[l−1] = x ⇔ **p[l−1] = p[r] − x**

For each r, count earlier prefixes equal to p[r] − x:

```
map<long long, int> seen;  seen[0] = 1;   // empty prefix p[0]
long long p = 0, ans = 0;
for (int i = 0; i < n; i++) {
    p += a[i];
    auto it = seen.find(p - x);
    if (it != seen.end()) ans += it->second;
    seen[p]++;
}
```

CSES 1661 Subarray Sums II. O(n log n). Two pointers can't do this (Week 6)!

Counting by remainders: CSES 1662

Count subarrays whose sum is divisible by n.

- sum(l..r) ≡ 0 (mod n) ⇔ p[r] ≡ p[l−1] (mod n)
- Count pairs of equal remainders: for each r, add cnt[p[r] mod n]
- Negative values: `((p % n) + n) % n`!

```
vector<long long> cnt(n, 0); cnt[0] = 1;
long long p = 0, ans = 0;
for (auto x : a) {
    p = ((p + x) % n + n) % n;
    ans += cnt[p]++;
}
```

Max subarray sum via prefix minima

sum(l..r) = p[r] − p[l−1] → maximize by subtracting the **smallest earlier prefix**

```
long long best = LLONG_MIN, minPrefix = 0, p = 0;
for (auto x : a) {
    p += x;
    best = max(best, p - minPrefix);
    minPrefix = min(minPrefix, p);
}
```

CSES 1643 Maximum Subarray Sum (equivalent to Kadane's algorithm). O(n).

2D prefix sums

P[i][j] = sum of the rectangle (1,1)–(i,j)

```
P[i][j] = a[i][j] + P[i-1][j] + P[i][j-1] - P[i-1][j-1]
```

```
+-------+---+
|   D   | B |      P[i][j] = A + (B+D) + (C+D) - D
+-------+---+                    ^P[i-1][j] ^P[i][j-1] ^P[i-1][j-1]
|   C   | A |
+-------+---+   (A = a[i][j])
```

2D rectangle query

Sum of rectangle rows r1..r2, columns c1..c2:

```
S = P[r2][c2] - P[r1-1][c2] - P[r2][c1-1] + P[r1-1][c1-1]
```

Subtract the strip above and the strip to the left; the top-left corner was subtracted twice → add it back (**inclusion–exclusion**).

O(nm) build, O(1) per query. CSES 1652 Forest Queries.

Forest Queries — code

```cpp
#include <bits/stdc++.h>
using namespace std;
int P[1001][1001];
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q; cin >> n >> q;
    for (int i = 1; i <= n; i++) {
        string row; cin >> row;
        for (int j = 1; j <= n; j++)
            P[i][j] = (row[j-1] == '*') + P[i-1][j] + P[i][j-1] - P[i-1][j-1];
    }
    while (q--) {
        int y1, x1, y2, x2; cin >> y1 >> x1 >> y2 >> x2;
        cout << P[y2][x2] - P[y1-1][x2] - P[y2][x1-1] + P[y1-1][x1-1] << '\n';
    }
}
```

Difference arrays: the reverse problem

Many updates "add v to a[l..r]", then read the final array once.

d[i] = a[i] − a[i−1]. Adding v to a[l..r] changes only two entries:

```
d[l]   += v
d[r+1] -= v
```

At the end, a = prefix sums of d. O(1) per update, O(n) to rebuild.

Difference array trace

n = 5, updates: +2 on [2,4], +3 on [1,2]

```
index:      1  2  3  4  5  6
start d:    0  0  0  0  0  0
+2 [2,4]:   0  2  0  0 -2  0     d[2] += 2, d[5] -= 2
+3 [1,2]:   3  2 -3  0 -2  0     d[1] += 3, d[3] -= 3
prefix a:   3  5  2  2  0
```

Check: a[2] = 2 + 3 = 5 ✓, a[3] = 2 ✓, a[5] = 0 ✓

Example: CF 816B — Karen and Coffee

n recipes recommend temperatures [l_i, r_i]. A temperature is admissible if ≥ k recipes recommend it. q queries: how many admissible temperatures in [a, b]?

1. Difference array over temperatures 1..200000 → coverage count per temperature
2. 0/1 array: admissible?
3. Prefix sums of that → each query O(1)

Difference array, then prefix sum — the two tools compose.

2D difference arrays

Add v to rectangle (r1,c1)–(r2,c2):

```
D[r1][c1]     += v
D[r1][c2+1]   -= v
D[r2+1][c1]   -= v
D[r2+1][c2+1] += v
```

Then take 2D prefix sums of D to get the final grid. O(1) per update, O(nm) total.

Higher order: CF 295A — Greg and Array

m operations "add d_i to a[l_i..r_i]"; k queries "apply operations x..y".

1. Difference array over **operations** → how many times each op is applied
2. Difference array over the **array** with weight (times × d_i)
3. Prefix sums → final array

O(n + m + k). Stacking difference arrays handles "updates of updates".

Complexity & limits

Task	Build	Query/Update
1D range sum	O(n)	O(1)
2D rectangle sum	O(nm)	O(1)
Range add, read at end	—	O(1) each + O(n) final
Count subarrays sum = x	O(n log n) with map	—

Updates **interleaved** with queries? Prefix sums break → Fenwick tree / segment tree (Weeks 23–24).

Pitfalls

- Forgetting p[0] = 0 → special case for l = 1, often wrong
- `int` overflow in sums → `long long`
- Negative remainders: `x % n` can be negative in C++
- 2D: mixing up row/column order in the query (CSES 1652 gives y then x)
- Difference array needs size n + 2 (writes to r + 1)
- `seen[0] = 1` forgotten → misses subarrays starting at index 1

Practice problems

- CSES 1646 Static Range Sum Queries · 1650 Range Xor Queries · 1652 Forest Queries · 1661 Subarray Sums II · 1662 Subarray Divisibility · 1643 Maximum Subarray Sum
- Codeforces 313B Ilya and Queries · 433B Kuriyama Mirai's Stones · 816B Karen and Coffee · 295A Greg and Array
- AtCoder ABC014 C AtColor
- USACO 2016 Jan Silver — Subsequences Summing to Sevens · 2015 Dec Silver — Breed Counting

Summary

- Prefix sums: O(n) build, O(1) range query, for any invertible operation
- "Subarray with property" ⇔ "pair of prefixes with property" → count with a hash map
- 2D prefix sums via inclusion–exclusion
- Difference arrays: O(1) range updates, rebuild with one prefix-sum pass
- Next week: **recursion & backtracking**